Name bhavya rana
Sapid 590034689
Batch 15

C program

```
#include <stdio.h>

#define MAX 10

// Recursive function to read matrix elements
void readMatrix(int matrix[MAX][MAX], int n, int row, int
col)
{
    if (row == n)
        return;

    scanf("%d", &matrix[row][col]);

    if (col == n - 1)
        readMatrix(matrix, n, row + 1, 0);
    else
        readMatrix(matrix, n, row, col + 1);
}

// Recursive function to multiply matrices
void multiplyMatrix(int A[MAX][MAX], int B[MAX][MAX],
                    int C[MAX][MAX], int n,
                    int i, int j, int k)
{
    // All rows completed
    if (i == n)
        return;

    // Calculate C[i][j]
    if (k < n)
```

```
    {
        C[i][j] += A[i][k] * B[k][j];
        multiplyMatrix(A, B, C, n, i, j, k + 1);
    }
    else
    {
        // Move to next column
        if (j < n - 1)
            multiplyMatrix(A, B, C, n, i, j + 1, 0);
        else
            // Move to next row
            multiplyMatrix(A, B, C, n, i + 1, 0, 0);
    }
}

// Recursive function to display matrix
void displayMatrix(int matrix[MAX][MAX], int n, int row, int
col)
{
    if (row == n)
        return;

    printf("%d ", matrix[row][col]);

    if (col == n - 1)
    {
        printf("\n");
        displayMatrix(matrix, n, row + 1, 0);
    }
    else
    {
        displayMatrix(matrix, n, row, col + 1);
    }
}

int main()
{
    int n;
    int A[MAX][MAX], B[MAX][MAX];
    int C[MAX][MAX] = {0};

    printf("Enter order of square matrices: ");
    scanf("%d", &n);

    printf("Enter elements of first matrix:\n");
```

```
    readMatrix(A, n, 0, 0);

    printf("Enter elements of second matrix:\n");
    readMatrix(B, n, 0, 0);

    multiplyMatrix(A, B, C, n, 0, 0, 0);

    printf("Product matrix:\n");
    displayMatrix(C, n, 0, 0);

    return 0;
}
```

Algorithm

1. Read the order `n` of the two square matrices.
2. Read matrices `A` and `B` recursively.
3. Initialize result matrix `C` with `0`.
4. Call `multiplyMatrix(A, B, C, n, 0, 0, 0)`.
5. For each element `C[i][j]`, recursively calculate:
   `C[i][j] = A[i][0]×B[0][j] + A[i][1]×B[1][j] + ...`
6. When `k == n`, move recursively to the next column.
7. After all columns are processed, move to the next row.
8. Stop when all `n` rows have been processed.
9. Display the resulting matrix recursively.

Test Case Table – Recursive Matrix Multiplication

Test Case	Matrix Size n	Matrix A	Matrix B	Expected Product Matrix
1	2	`1 2` `3 4`	`5 6` `7 8`	`19 22` `43 50`
2	2	`1 0` `0 1`	`2 3` `4 5`	`2 3` `4 5`
3	2	`0 0` `0 0`	`1 2` `3 4`	`0 0` `0 0`
4	1	`5`	`4`	`20`
5	2	`-1 2` `3 -4`	`2 1` `-3 5`	`-8 9` `18 -17`

6	3	`1 2 3`
		`4 5 6`
		`7 8 9`
			`9 8 7`
			`6 5 4`
			`3 2 1`
				`30 24 18`
				`84 69 54`
				`138 114 90`

```
Last login: Thu Nov  5 21:17:00 on ttys000
[bhavya@Bhavyas-MacBook-Air ~ % nano day66.c
[bhavya@Bhavyas-MacBook-Air ~ % clang day66.c -o day66
[bhavya@Bhavyas-MacBook-Air ~ % ./day66
Enter order of square matrices: 2
Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8
Product matrix:
19 22
43 50
[bhavya@Bhavyas-MacBook-Air ~ % ./day66
```

Execution